EAST WEST UNIVERSITY

Project Report

The Whimsical Widget Workshop

CSE325: Operating Systems
Section: 01
Semester: Fall-25

Group No	04
Project No	13

Submitted To:
Shatabdi Roy Moon,
Lecturer, Department of CSE
East West University

Submitted by:

Student ID	Student Name	Contribution Percentage
2023-3-60-193	Ahnaf Dewan	25%
2023-3-60-504	Md Mahiruddin Chowdhury	25%
2023-3-60-618	Bushramoni Neha	25%
2023-3-60-271	Esrat Jahan Jerin	25%

Submission Date: **18 December, 2025**

Problem Statement:

In many computer programs, more than one task runs at the same time. These tasks are called threads. When several threads run together, they often need to use the same things, such as files, tools, or data. These shared things are called resources.

The problem starts when many threads try to use the same resource at the same time. If this is not handled properly then two threads may use the same resource together, which can cause errors. Some threads may never get a chance to use the resource and even the program may behave differently every time it runs.There is also high chance that the system may become slow or stuck.

This project tries to solve this problem using a simple example. It imagines a small company called **Whimsical Widget Workshop**. In this company:

- Each engineer is a thread
- Each tool is a shared resource
- Only one engineer can use a tool at a time

Each engineer needs one or more tools to make a widget. Engineers work at the same time, and they ask for tools whenever they need them. The main challenge is to make sure that:

- No two engineers use the same tool at the same time
- Engineers get tools in the order they ask for them
- No engineer has to wait forever
- The program runs correctly even when timing changes

This problem is similar to real computer systems where many programs run together and share limited resources.

Introduction:

Our project aims to solve the above problems for the Whimsical Widget Workshop where several engineers work concurrently to build widgets. Each engineer performs tasks independently, but during the production process they must use limited shared resources such as design assets, coding templates, and testing tools. Since these resources are limited, only one engineer is allowed to use a particular resource at any given time. Additionally, fairness is required so that resources are allocated in the order engineers request them, ensuring that no engineer waits indefinitely.

Our project uses few critical thread libraries to tackle the issues. The first and foremost, we are using semaphores to enforce mutual exclusion on shared resources such as “Coding Template”, “Design Assets” and “Testing Tools”. However, these resources are limited. The Engineers like “Alice” “Bob” and “Carol” are represented as Threads working in parallel. To ensure this equal access based on waiting time, each resource is placed in a FIFO (First In First Out) watinng queue. To safeguard the shared data, here the resources and guarantee proper thread synchronization, a mutex lock is used. This ensires that all engineers eventually findih executing their assigned widgets and avoids deadlocks and prevent the race condition.

To demonstrate further, we can image Alice wants to work on a widget that needs a coding tempolte and a testing tool, while Bob needs to work on another widget needing the same testing tool “resource”. If Alice starts first, Bob must wait until Alice releases the testing tool. This ensures that both engineers complete their tasks safely and fairly.

By combining the mechanisms like semaphores, mutex locks, FIFO queue we can ensure that all engineers can work efficiently on their widgets without conflicting or without any deadlock mainting smooth operation.

Techniques Used:

Semaphores: A semaphore is a synchronization tool in computing that controls access to shared resources by multiple processes/threads, preventing chaos like race conditions. Semaphores are of 2 types i) Binary Semaphores (values 1 or 0) ii) Counting Semaphores (values 0 to n)

In our program, each resource has its own semaphores like:

resource_sem[0] → Design Assets

resource_sem[1] → Coding Templates

resource_sem[2] → Testing Tools

where each semaphore, is initialized with a value of **1**, representing that only one engineer can use that resource at a time. It ensures that engineers do not simultaneously access the same resource, preventing data conflicts or logical errors.

Functions used are:

Function	**Purpose in Project**
sem_init(&resource_sem[i], 0, 1)	Initialize the semaphore for each resource to **1**, indicating one available unit.
sem_wait(&resource_sem[i])	Called when an engineer requests a resource. If the resource is unavailable, the engineer waits.
sem_post(&resource_sem[i])	Called when an engineer releases a resource, allowing the next waiting engineer to acquire it.
sem_destroy(&resource_sem[i])	Cleans up semaphores at the end of the program.

FIFO Queue:

To ensure fairness, each resource is associated with a FIFO (First-In, First-Out) queue. When an engineer requests a resource, their ID is added to the queue.Only the engineer at the front of the queue can acquire the semaphore. Others must wait until their turn arrives.

Mutex Locks:

A mutex (mutual exclusion) is a programming tool that acts like a lock, ensuring only *one* thread can access a shared resource (like a piece of data or code section) at a time, preventing data corruption from simultaneous access (race conditions) in multi-threaded programs. It works by a thread acquiring the lock (lock the resource) before entering a "critical section," and releasing it (unlock) only after it finishes, forcing other threads to wait until the resource is free.

One example is,

- Alice releases the Testing Tool → mutex locks → Bob dequeues → semaphore acquired → mutex unlocks.
- This avoids two threads trying to update the queue simultaneously.

PseudoCode:

```
Start
Initialize semaphores for all resources (Design, Code, Test)
Initialize FIFO queues for each resource
Initialize mutex lock for queues and logging

Create threads for each Engineer (Alice, Bob, Carol)
For each Engineer Thread:
   For each Widget assigned to the Engineer:

      Randomly select 1–2 required resources
      Add Engineer to FIFO waiting queue for each required resource
      Wait until it is the Engineer's turn in the queue
      Acquire required resources using semaphores
      Build widget (simulate work with sleep)
      Release resources using semaphores

Wait for all Engineer threads to complete
Destroy semaphores and mutex lock
```

End

Output of the Code:

```
vboxuser@ubuntu: ~/project
vboxuser@ubuntu:~/project$ gcc project.c -o project
vboxuser@ubuntu:~/project$ ./project
==========================================
  WHIMSICAL WIDGET WORKSHOP OPENED
==========================================

[WAITING] Alice wants widget 1 and needs: Coding Templates
Alice is requesting Coding Templates
Alice acquired Coding Templates
[WORKING] Alice is working on widget 1

[WAITING] Bob wants widget 1 and needs: Design Assets Coding Templates
Bob is requesting Design Assets
Bob acquired Design Assets
Bob is requesting Coding Templates

[WAITING] Carol wants widget 1 and needs: Design Assets
Carol is requesting Design Assets
[DONE] Alice finished widget 1
Alice released Coding Templates
Bob acquired Coding Templates
[WORKING] Bob is working on widget 1
```

```
vboxuser@ubuntu: ~/project
[WORKING] Bob is working on widget 1

[WAITING] Alice wants widget 2 and needs: Design Assets Testing Tools
Alice is requesting Design Assets
[DONE] Bob finished widget 1
Bob released Design Assets
Bob released Coding Templates
Carol acquired Design Assets
[WORKING] Carol is working on widget 1

[WAITING] Bob wants widget 2 and needs: Design Assets Coding Templates
Bob is requesting Design Assets
[DONE] Carol finished widget 1
Carol released Design Assets
Alice acquired Design Assets
Alice is requesting Testing Tools
Alice acquired Testing Tools
[WORKING] Alice is working on widget 2

[WAITING] Carol wants widget 2 and needs: Design Assets
Carol is requesting Design Assets
[DONE] Alice finished widget 2
Alice released Design Assets
Alice released Testing Tools
```

vboxuser@ubuntu: ~/project

```
Alice is requesting Testing Tools
Alice acquired Testing Tools
[WORKING] Alice is working on widget 2

[WAITING] Carol wants widget 2 and needs: Design Assets
Carol is requesting Design Assets
[DONE] Alice finished widget 2
Alice released Design Assets
Alice released Testing Tools
Bob acquired Design Assets
Bob is requesting Coding Templates
Bob acquired Coding Templates
[WORKING] Bob is working on widget 2

[WAITING] Alice wants widget 3 and needs: Testing Tools
Alice is requesting Testing Tools
Alice acquired Testing Tools
[WORKING] Alice is working on widget 3
[DONE] Bob finished widget 2
Bob released Design Assets
Bob released Coding Templates
Carol acquired Design Assets
[WORKING] Carol is working on widget 2
```

vboxuser@ubuntu: ~/project

```
Carol acquired Design Assets
[WORKING] Carol is working on widget 2

[WAITING] Bob wants widget 3 and needs: Design Assets Testing Tools
Bob is requesting Design Assets
[DONE] Carol finished widget 2
Carol released Design Assets
Bob acquired Design Assets
Bob is requesting Testing Tools

[WAITING] Carol wants widget 3 and needs: Coding Templates Testing Tools
Carol is requesting Coding Templates
Carol acquired Coding Templates
Carol is requesting Testing Tools
[DONE] Alice finished widget 3
Alice released Testing Tools
Bob acquired Testing Tools
[WORKING] Bob is working on widget 3
[DONE] Bob finished widget 3
Bob released Design Assets
Bob released Testing Tools
Carol acquired Testing Tools
[WORKING] Carol is working on widget 3
[DONE] Carol finished widget 3
```

```
[WAITING] Carol wants widget 3 and needs: Coding Templates Testing Tools
Carol is requesting Coding Templates
Carol acquired Coding Templates
Carol is requesting Testing Tools
[DONE] Alice finished widget 3
Alice released Testing Tools
Bob acquired Testing Tools
[WORKING] Bob is working on widget 3
[DONE] Bob finished widget 3
Bob released Design Assets
Bob released Testing Tools
Carol acquired Testing Tools
[WORKING] Carol is working on widget 3
[DONE] Carol finished widget 3
Carol released Coding Templates
Carol released Testing Tools

============================================
  ALL WIDGETS COMPLETED — WORKSHOP CLOSED
============================================
vboxuser@ubuntu:~/project$
```

Conclusion:

In conclusion, this project clearly demonstrates how multiple threads can operate simultaneously while sharing limited resources in a safe and fair manner. By viewing engineers as threads and tools as shared resources, the program effectively illustrates how real operating systems manage multitasking. The program uses semaphores to ensure that only one engineer can access a resource at any given moment. Mutex locks safeguard shared data from being changed incorrectly. Waiting queues help maintain order, allowing engineers to receive resources in the same sequence they request them, so no one has to wait indefinitely. Although the work and timing of each engineer are random, the program runs smoothly without crashes or conflicts. Overall, this project lays a solid foundation for understanding thread synchronization, resource management, and fairness. These concepts are crucial in real-world systems like operating systems, servers, and multi-user applications, making this project a valuable learning experience.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define NUM_ENGINEERS 3
#define NUM_WIDGETS   3
#define NUM_RESOURCES 3

#define DESIGN  0
#define CODE    1
#define TEST    2

typedef struct {
    int id;
    char* name;
} engineer_t;
char* resource_names[NUM_RESOURCES] = {
    "Design Assets",
    "Coding Templates",
    "Testing Tools"
};
char* engineer_names[NUM_ENGINEERS] = {
    "Alice",
    "Bob",
    "Carol"
};
// One semaphore per resource (only 1 available)
sem_t resource_sem[NUM_RESOURCES];
```

```
// FIFO queues for fairness
int waiting_queue[NUM_RESOURCES][NUM_ENGINEERS];
int front[NUM_RESOURCES] = {0};
int rear[NUM_RESOURCES]  = {0};
// Mutex for queues and logging
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void enqueue(int res, int eng) {
    waiting_queue[res][rear[res]] = eng;
    rear[res] = (rear[res] + 1) % NUM_ENGINEERS;
}
int dequeue(int res) {
    if (front[res] == rear[res])
        return -1;
    int id = waiting_queue[res][front[res]];
    front[res] = (front[res] + 1) % NUM_ENGINEERS;
    return id;
}
//the threads requesting the resources
void request_resource(int eng, int res) {
    pthread_mutex_lock(&mutex);
    printf("%s is requesting %s\n",
            engineer_names[eng], resource_names[res]);
    enqueue(res, eng);
    pthread_mutex_unlock(&mutex);
    //wait until it's engineer's turn
    while (1) {
        //wait for resource to be free -> wait until nobody is using this resource
        sem_wait(&resource_sem[res]);
        pthread_mutex_lock(&mutex);
        int next = dequeue(res);
```

```
        if (next == eng) {
            printf("%s acquired %s\n",
                    engineer_names[eng], resource_names[res]);
            pthread_mutex_unlock(&mutex);
            return;
        }
        // Not this engineer's turn
        //we removed someone else so we put them back in front order (fairness)
        enqueue(res, next);
        pthread_mutex_unlock(&mutex);
        sem_post(&resource_sem[res]);   //give this resource back
        sleep(1);
    }
}
void release_resource(int eng, int res) {
    pthread_mutex_lock(&mutex);
    printf("%s released %s\n",
            engineer_names[eng], resource_names[res]);
    pthread_mutex_unlock(&mutex);

    sem_post(&resource_sem[res]);
}

void* engineer_thread(void* arg) {
    engineer_t* e = (engineer_t*)arg;

    for (int w = 0; w < NUM_WIDGETS; w++) {
        // Randomly choose 1–2 resources
        int need[NUM_RESOURCES] = {0};
        int count = rand() % 2 + 1;

        for (int i = 0; i < count; i++) {
```

```
    int r;
    do { r = rand() % NUM_RESOURCES; } while (need[r]);
    need[r] = 1;
}

pthread_mutex_lock(&mutex);
printf("\n[WAITING] %s wants widget %d and needs:",
        e->name, w + 1);
for (int r = 0; r < NUM_RESOURCES; r++)
    if (need[r]) printf(" %s", resource_names[r]);
printf("\n");
pthread_mutex_unlock(&mutex);
// requesting for resources
for (int r = 0; r < NUM_RESOURCES; r++)
    if (need[r]) request_resource(e->id, r);
// Our threads are working
pthread_mutex_lock(&mutex);
printf("[WORKING] %s is working on widget %d\n",
        e->name, w + 1);
pthread_mutex_unlock(&mutex);

sleep(rand() % 3 + 1);

pthread_mutex_lock(&mutex);
printf("[DONE] %s finished widget %d\n",
        e->name, w + 1);
pthread_mutex_unlock(&mutex);

// Release the resources
for (int r = 0; r < NUM_RESOURCES; r++)
    if (need[r]) release_resource(e->id, r);
```

```
        sleep(1);
    }
    return NULL;
}
int main() {
    srand(time(NULL));
    pthread_t threads[NUM_ENGINEERS];
    engineer_t engineers[NUM_ENGINEERS] = {
        {0, "Alice"},
        {1, "Bob"},
        {2, "Carol"}
    };
    // Initializing semaphores
    for (int i = 0; i < NUM_RESOURCES; i++)
        sem_init(&resource_sem[i], 0, 1);
    printf("============================================\n");
    printf("  WHIMSICAL WIDGET WORKSHOP OPENED\n");
    printf("============================================\n");
    // Creating engineer threads
    for (int i = 0; i < NUM_ENGINEERS; i++)
        pthread_create(&threads[i], NULL, engineer_thread, &engineers[i]);
    // Waiting for all engineers
    for (int i = 0; i < NUM_ENGINEERS; i++)
        pthread_join(threads[i], NULL);
    // Cleaning
    for (int i = 0; i < NUM_RESOURCES; i++)
        sem_destroy(&resource_sem[i]);
    printf("\n============================================\n");
    printf("  ALL WIDGETS COMPLETED — WORKSHOP CLOSED\n");
    printf("============================================\n");
    return 0;
}
```